```
show_regexp(a, /red (ball|angry) sky/) # => the ->red angry sky<-
```

Parentheses do double duty in regular expressions, they also collect the results of pattern matching. Ruby counts opening parentheses and for each stores the result of the partial match between it and the corresponding closing parenthesis. You can use this partial match both within the rest of the pattern and in your Ruby program. Within the pattern, the sequence \1 refers to the match of the first group, \2 the second group, and so on. Outside the pattern, the special global variables $1, $2, and so on, serve the same purpose, and are reset on every regular expression match, just like $~.

```
/(\d\d):(\d\d)(..)/ =~ "12:50am"   # => 0
"Hour is #$1, minute #$2"          # => "Hour is 12, minute 50"
/((\d\d):(\d\d))(..)/ =~ "12:50am" # => 0
"Time is #$1"                      # => "Time is 12:50"
"Hour is #$2, minute #$3"          # => "Hour is 12, minute 50"
"AM/PM is #$4"                     # => "AM/PM is am"
```

If you're using the MatchData object returned by the match method, you can index into it to get the corresponding subpatterns. This is much more common than using the global magic variables.

```
md = /(\d\d):(\d\d)(..)/.match("12:50am")
"Hour is #{md[1]}, minute #{md[2]}" # => "Hour is 12, minute 50"
md = /((\d\d):(\d\d))(..)/.match("12:50am")
"Time is #{md[1]}"                  # => "Time is 12:50"
"Hour is #{md[2]}, minute #{md[3]}" # => "Hour is 12, minute 50"
"AM/PM is #{md[4]}"                 # => "AM/PM is am"
```

The ability to use part of the current match later in that match allows you to look for various forms of repetition:

```
  # match duplicated letter
show_regexp('He said "Hello"', /(\w)\1/) # => He said "He->ll<-o"
  # match duplicated substrings
show_regexp('Mississippi', /(\w+)\1/)    # => M->ississ<-ippi
```

Rather than use numbers, you can use names to refer to previously matched content. You give a group a name by placing ?<_name_> immediately after the opening parenthesis. You can subsequently refer to this named group using \k<_name_> (or \k'_name_').

tut_regexp/named_regex_groups_1.rb

```
 # match duplicated letter
str = 'He said "Hello"'
show_regexp(str, /(?<char>\w)\k<char>/)  # => He said "He->ll<-o"

 # match duplicated adjacent substrings
str = "Mississippi"
show_regexp(str, /(?<seq>\w+)\k<seq>/)   # => M->ississ<-ippi
```

The named matches in a regular expression are also available as local variables, but only if you use a literal regexp and that literal appears on the left hand side of the =~ operator. (So you can't assign a regular expression object to a variable, match the contents of that variable against a string, and expect the local variables to be set.)

tut_regexp/named_regex_groups_2.rb

```
/(?<hour>\d\d):(?<min>\d\d)(..)/ =~ "12:50am"  # => 0
```